

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Name	Abinesh M
Register Number	192465050
Date	
Time	1 Hr

COURSE NAME: **Cybersecurity Incident Handling and Response**
COURSE CODE: **CSA60**

COURSE OUTCOME COVERED

C04: Appraise containment, eradication, recovery strategies by selecting appropriate techniques such as isolation, malware removal, patch management, system restoration to ensure secure operations. (BL3, BL4)

Activity Tool : Malware Eradication and System Hardening (Medium)
Assessment Tool Weightage: 20%

Code of Conduct:

I Abinesh M 192465050 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Abinesh M
192465050

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Malware Detection and Eradication	25%	Thoroughly identifies malware, malicious processes, files, and persistence mechanisms and completely removes the threat with proper verification.	Identifies and removes most malware components with minor omissions.	Performs basic malware removal but misses some malicious components or persistence mechanisms.	Unable to identify or remove the malware effectively.
Vulnerability Remediation and Patch Management	20%	Accurately identifies exploited vulnerabilities, applies appropriate patches or fixes, and verifies that vulnerabilities are fully remediated.	Identifies and remediates major vulnerabilities with minor gaps in verification.	Performs basic patching or remediation but misses important vulnerabilities.	Unable to identify or remediate relevant vulnerabilities.
System and Network Hardening	25%	Effectively applies comprehensive hardening techniques including secure configurations, least privilege, firewall controls, service reduction, authentication, and network security.	Applies appropriate hardening techniques with minor configuration gaps.	Implements basic hardening but leaves significant security weaknesses.	Unable to apply appropriate system or network hardening measures.
Recovery and Security Validation	15%	Successfully restores affected systems, validates system integrity, verifies malware removal, and performs thorough post-recovery security checks.	Restores and validates systems with minor omissions.	Performs basic recovery and validation but misses important security checks.	Unable to safely restore or validate the recovered systems.
Documentation and Incident Report	15%	Provides clear, professional documentation covering malware analysis, eradication, vulnerabilities, hardening actions, validation results, evidence, and conclusions.	Documentation is well organized with most required evidence and findings.	Documentation is partially organized with limited evidence or analysis.	Documentation is incomplete, poorly organized, and lacks supporting evidence.

Malware Eradication and Vulnerability Remediation

Question 1: Malware Eradication and System Cleaning

Malware detected on multiple employee systems, accompanied by suspicious processes and unauthorized network connections, indicates an active infection that has likely spread through the environment via a common vector — phishing, a compromised removable device, or a network share. As the incident responder, my priority is to move through a disciplined sequence: identify the full scope of infection, isolate affected systems to stop further spread, remove the malware and everything it left behind, and verify the environment is genuinely clean before returning systems to production.

1. Identification

- **Confirm and scope the infection:** Use EDR/antivirus alerts, the suspicious process names, and unauthorized network connections already identified as a starting point, then search across all endpoints for the same process names, file hashes, or destination IPs to determine the full list of affected systems.
- **Capture volatile evidence first:** Before touching an infected system, capture running processes, active network connections, and memory (where feasible) for forensic analysis, since containment actions like killing processes or disconnecting the network will destroy this evidence.
- **Identify the malware family and behavior:** Submit collected samples/hashes to a sandbox and reputation services (VirusTotal, internal sandbox) to understand what the malware does — persistence mechanism, C2 behavior, propagation method — which shapes the cleanup plan.
- **Determine the infection vector:** Trace back through email, web-proxy, or removable-media logs to establish patient zero and how the malware initially entered the environment, since eradication is incomplete if the entry point is not also addressed.

2. Isolation

- Immediately isolate confirmed infected hosts from the network — via EDR network-containment, disabling the switch port, or removing from Wi-Fi — while keeping the system powered on for forensic capture.
- Disable or reset credentials for any accounts active on infected systems, since malware with keylogging or credential-theft capability may have already captured them.
- Block identified C2 IPs/domains and malicious file hashes at the firewall, proxy, and EDR platform organization-wide to prevent the malware from communicating or spreading further while cleanup is underway.
- Segment or restrict network shares and lateral-movement paths (e.g., disable unnecessary SMB access, restrict RDP) to limit the malware's ability to spread to additional systems during the response.

3. Removal (Eradication)

- **Terminate malicious processes:** End all identified malicious processes across affected hosts, using EDR tooling for controlled termination rather than manual task-killing, which can trigger destructive behavior in some malware.
- **Remove malicious files and artifacts:** Delete the malware binaries, dropped secondary payloads, and any files created during execution, cross-referencing the sandbox report to ensure nothing is missed.
- **Eliminate persistence mechanisms:** Check and clean registry Run keys, scheduled tasks, new services, startup folder entries, WMI event subscriptions, and any new local/domain accounts the malware may have created to survive a reboot.
- **Prefer reimaging where feasible:** For systems where full eradication cannot be confidently confirmed via cleanup alone, rebuild from a known-good image rather than risk leaving a hidden persistence mechanism in place — this is often faster and more reliable than exhaustive manual cleaning.

4. Verification

Verification Check	What Confirms a Clean System
Full AV/EDR re-scan	No detections across all previously infected hosts using updated signatures
Process and persistence audit	No unauthorized processes, scheduled tasks, services, or registry run keys remain
Network monitoring	No further connections to known C2 infrastructure over a defined monitoring period
Log review	No recurrence of the original alert pattern across a full business cycle

Only after all verification checks pass are systems returned to normal production use. The investigation concludes with a post-incident report covering the infection vector, scope, timeline, and remediation, along with updated detection rules and any user-awareness or patching gaps identified along the way.

5. Tools and Techniques Supporting Eradication

- **EDR platforms (CrowdStrike Falcon, Microsoft Defender for Endpoint, SentinelOne):** provide remote process termination, file quarantine, and network isolation from a central console without needing physical access to each machine.
- **Forensic imaging tools (FTK Imager, dd, Velociraptor):** capture disk and memory images before remediation actions alter the system, preserving evidence for root-cause analysis and, if needed, legal proceedings.
- **Autoruns and persistence scanners (Sysinternals Autoruns, Wazuh, OSSEC):** enumerate every configured startup location — registry run keys, scheduled tasks, services, WMI subscriptions — so persistence mechanisms are not missed during manual review.

- **Sandbox/dynamic analysis (Any.Run, Joe Sandbox, Cuckoo):** safely detonate the malware sample to observe its full behavior, confirming which files, keys, and network destinations to search for across the environment.
- **Centralized deployment tools (SCCM, Intune, Ansible):** push cleanup scripts, updated signatures, or full reimaging jobs to many affected endpoints simultaneously, which is essential when the infection has spread to several systems.

6. Common Persistence Mechanisms to Check

Because malware authors rely on the same limited set of Windows and Linux persistence techniques, a thorough eradication checklist should explicitly verify each of the following before a system is declared clean:

Persistence Location	What to Check
Registry Run keys	HKLM/HKCU Run, RunOnce, and Winlogon keys for unrecognized entries
Scheduled tasks / cron jobs	Any task created around the infection timeframe, especially ones running from temp directories
Services	New or modified Windows services and Linux systemd units pointing to unfamiliar binaries
Startup folders	Shortcuts or scripts placed in user or all-users startup locations
Accounts and groups	New local/domain accounts or unexpected additions to privileged groups
Browser extensions	Unrecognized extensions that could re-download or re-trigger malicious payloads

Working through this checklist systematically — rather than relying solely on antivirus detection — is what catches the persistence mechanisms that signature-based tools sometimes miss, particularly for custom or fileless malware.

Question 2: Malware Removal and Vulnerability Remediation

An attacker exploiting an unpatched software vulnerability to infect a server represents a different challenge from user-driven infections: the entry point is a technical flaw rather than human action, and eradication is incomplete unless the vulnerability itself is also closed. As the responding analyst, my process addresses both the malware present on the system and the underlying weakness that allowed it in, since removing the malware alone leaves the server immediately re-exploitable.

1. Identifying the Root Cause

- **Analyze exploitation artifacts:** Review web server, application, and system logs around the time of compromise for exploitation signatures — unusual request patterns, error spikes, or process spawns immediately following a specific request — to pinpoint which service was targeted.
- **Correlate with known CVEs:** Match the affected software, version, and observed exploitation pattern against public vulnerability databases (NVD, vendor advisories) to identify the specific CVE being exploited.
- **Confirm via forensic timeline:** Establish the exact sequence — vulnerable service receives malicious request → exploit executes → malware dropped/executed → persistence established — to confirm the vulnerability (not a stolen credential or other vector) was the true entry point.
- **Assess blast radius:** Determine what the exploited service had access to (other internal systems, databases, credentials) to scope how far the attacker could plausibly have moved from this entry point.

2. Removing the Malicious Components

- Isolate the server from the network while preserving system state for forensic capture, following the same evidence-preservation principle as any other infected host.
- Identify and terminate any malicious processes, web shells, or backdoors the attacker deployed via the exploit — web shells in particular are common in vulnerability-based server compromises and must be searched for explicitly in web-accessible directories.
- Remove all dropped files, modified configuration files, and any new or altered accounts, scheduled tasks, or services created post-exploitation.
- Where the compromise is deep or persistence is uncertain, rebuild the server from a clean, patched image rather than attempting to fully clean an actively exploited production system — this is generally the safer and faster path for server-level compromises.

3. Applying Security Patches

- **Patch the specific vulnerability:** Apply the vendor-released patch or update that addresses the identified CVE, testing in a staging environment first where the change window allows, given this is a security-critical fix.
- **Apply compensating controls if a patch isn't yet available:** Where no patch exists yet, apply vendor-recommended mitigations (WAF rules, configuration changes, disabling the vulnerable feature/module) to close the exposure until a permanent fix ships.
- **Patch broadly, not just the one server:** Scan the environment for other systems running the same vulnerable software version and patch them proactively, since the attacker (or others) may target the same flaw elsewhere.
- **Review related configuration:** Harden the service's configuration — disable unnecessary features, apply least-privilege service accounts, and restrict network exposure — to reduce the impact of any future undiscovered vulnerability in the same component.

4. Verifying the Fix

- **Vulnerability re-scan:** Run an authenticated vulnerability scan against the patched server to confirm the specific CVE (and ideally the broader vulnerability class) no longer registers as exploitable.
- **Penetration/exploit re-test:** Where feasible, safely attempt to reproduce the original exploitation technique in a controlled manner to directly confirm the patch closes the specific attack path used.
- **Malware and persistence re-check:** Re-run full AV/EDR scans and a manual audit for web shells, backdoor accounts, and scheduled tasks to confirm no remnant of the attacker's access remains post-patch.
- **Monitor post-remediation:** Place the server under heightened log and network monitoring for a defined period after returning it to production, watching specifically for any attempt to re-exploit the same vulnerability or use previously observed C2 infrastructure.

Once patching, malware removal, and verification are all confirmed, the incident is formally closed with a report documenting the exploited CVE, the timeline of compromise, remediation actions taken, and any patch-management process gaps that allowed the vulnerability to remain unpatched — feeding directly into improved vulnerability-management practices going forward.

5. Tools and Techniques Supporting Remediation

- **Vulnerability scanners (Nessus, Qualys, OpenVAS):** identify the specific CVE and confirm patch status both before remediation (to scope affected systems) and after (to verify the fix).
- **Web application firewalls (WAF) and IPS:** provide a compensating control that can virtually patch a known exploit pattern while the permanent vendor fix is tested and scheduled for deployment.
- **Patch management platforms (WSUS, SCCM, Ansible, Puppet):** deploy the vendor patch consistently across all affected systems and provide auditable confirmation of successful installation, avoiding the risk of a manually patched server being missed elsewhere in the fleet.
- **Web shell detection tools (web server log analysis, YARA rules, file-integrity monitoring):** specifically search web-accessible directories for backdoor scripts dropped via the exploited vulnerability, since these are easy to overlook amid normal application files.
- **Configuration management / hardening baselines (CIS Benchmarks):** provide a reference standard for re-hardening the server's configuration alongside the patch, reducing the chance a related but distinct weakness is exploited next.

6. Example Exploitation-to-Remediation Timeline

A representative timeline illustrates how root-cause identification, eradication, and patching typically unfold together for a server-side exploit:

Phase	Typical Actions
Detection	EDR/WAF alert on anomalous process spawned by the web service; server isolated pending investigation
Root-cause analysis	Log review identifies the malicious request pattern; matched to a public CVE for the running software version
Eradication	Web shell and dropped payloads located and removed; server rebuilt from a clean image where persistence is uncertain
Remediation	Vendor patch applied and tested; WAF rule added as a compensating control during the patch window
Verification	Authenticated vulnerability scan confirms the CVE no longer applies; server returned to production under heightened monitoring

This phased approach ensures the server is not returned to production until both the immediate compromise and its root technical cause have been independently confirmed as resolved — preventing the common failure mode of cleaning a server only to have it re-exploited through the same unpatched flaw days later.

Conclusion

Both scenarios follow the same underlying eradication discipline — identify the full scope, isolate to prevent further damage, remove every malicious component including persistence mechanisms, and verify cleanliness before returning systems to service — but they diverge in where the root cause lies. User-driven infections require closing a behavioral or process gap (a phishing vector, missing endpoint controls), while exploit-driven server compromises require closing a technical gap through patching and configuration hardening. In both cases, eradication is only complete when the entry point itself is addressed; removing the malware while leaving the underlying weakness in place — whether a training gap or an unpatched CVE — simply invites reinfection.

Comparing the Two Scenarios

Aspect	Question 1: Endpoint Malware	Question 2: Server Exploitation
Entry point	User-driven vector (phishing, media, share)	Unpatched software vulnerability (CVE)
Root-cause fix	User awareness, email/endpoint controls	Patch deployment and configuration hardening
Typical artifacts	Malware binaries, C2 processes, persistence keys	Web shells, backdoor accounts, exploit payloads

Aspect	Question 1: Endpoint Malware	Question 2: Server Exploitation
Verification focus	Clean AV/EDR scan; no recurring alert pattern	Vulnerability re-scan; confirmed patch effectiveness

Regardless of entry point, the response benefits from the same evidence-first discipline: preserve volatile data before containment, confirm scope before declaring an incident closed, and feed lessons learned — whether about phishing susceptibility or patch-management gaps — back into the organization's broader security program to reduce the likelihood of a repeat incident.